

PRACTICAL 10

Create a simple permissioned blockchain using Hyperledger Fabric

I. CONCEPTS

What does “permissioned blockchain” mean?

Public Blockchain	Permissioned Blockchain
Anyone can join	Only approved members
Anonymous	Known identities
Bitcoin / Ethereum	Hyperledger Fabric

Hyperledger Fabric is used by **companies**, not public crypto users. Generally, Hyperledger Fabric runs using **Docker + Linux environment**. So, we will use Windows + WSL (Ubuntu).

What is Hyperledger Fabric?

Hyperledger Fabric is: Permissioned blockchain, Identity-based, No mining, No cryptocurrency, Used by enterprises

Unlike Ethereum:

- Ethereum → Public
- Fabric → Private (only authorized members)

When we start the Fabric network, the following components are created:

1. Peer:

- A node in the network
- Stores ledger copy
- Executes smart contracts

Think of it as a computer that participates in blockchain

In test-network:

- Org1 Peer
- Org2 Peer

2. Orderer:

- Collects transactions
- Orders them into blocks
- Sends blocks to peers

In simple words: It decides transaction order. No mining involved.

3. Channel:

- A private communication layer
- Only specific organizations can join

Think of it as a private WhatsApp group inside blockchain

4. Chaincode (Smart Contract): In Fabric, smart contracts are called **Chaincode**. Chaincode defines business logic as well as creates and updates ledger data. **Example:** Asset transfer contract

II. INSTALLATION

1. **WSL** Windows Subsystem for Linux (WSL) is needed for Hyperledger Fabric development on Windows because the Fabric documentation and samples rely heavily on a native Linux Bash environment and its associated tools. WSL2 provides a high-performance, native Linux kernel environment directly within Windows, which is the recommended path for development. The Hyperledger Fabric samples and documentation primarily use bash scripts (e.g., `install-fabric.sh`, `network.sh`) for tasks like installing prerequisites, spinning up the network, and running commands. These scripts are native to Linux and would not run correctly in a standard Windows command prompt or PowerShell without significant modification or a compatibility layer.

Install: Open Terminal/Windows PowerShell → Type “`wsl --install`”. You will see following screens.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shikh> wsl --install
Downloading: Ubuntu
[=====60.8%=====]
```

```
C:\WINDOWS\System32\Win...
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shikh> wsl --install
Downloading: Ubuntu
Installing: Ubuntu
Distribution successfully installed. It can be launched via 'wsl.exe -d Ubuntu'
Launching Ubuntu...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: shikh
```

Now, click/press Enter → Type “New password”, say “*pass*” → Then “Retype new password”, say “*pass*”. You will see following screen.

```
shikh@Shikha: /mnt/c/Users/...
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shikh> wsl --install
Downloading: Ubuntu
Installing: Ubuntu
Distribution successfully installed. It can be launched via 'wsl.exe -d Ubuntu'
Launching Ubuntu...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: shikh
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

shikh@Shikha: /mnt/c/Users/shikh$ |
```

Verify WSL2: Close the Terminal and re-open. Run “*wsl --list --verbose*” in Terminal/Windows PowerShell to verify if WSL2 has been installed.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shikh> wsl --list --verbose
NAME        STATE          VERSION
* Ubuntu    Stopped        2
PS C:\Users\shikh> |
```

2. Docker Desktop (Most important tool) (It is a required prerequisite software for development and testing environments on Windows and macOS, providing the underlying [Docker Engine](#) and Docker Compose tools to run network components in containers.) [Hyperledger Fabric is designed to run its various components—such as peer nodes, ordering service nodes (orderers), and Certificate Authorities (CAs)—as isolated Docker containers. Docker Desktop manages these containers.)]

Why needed?

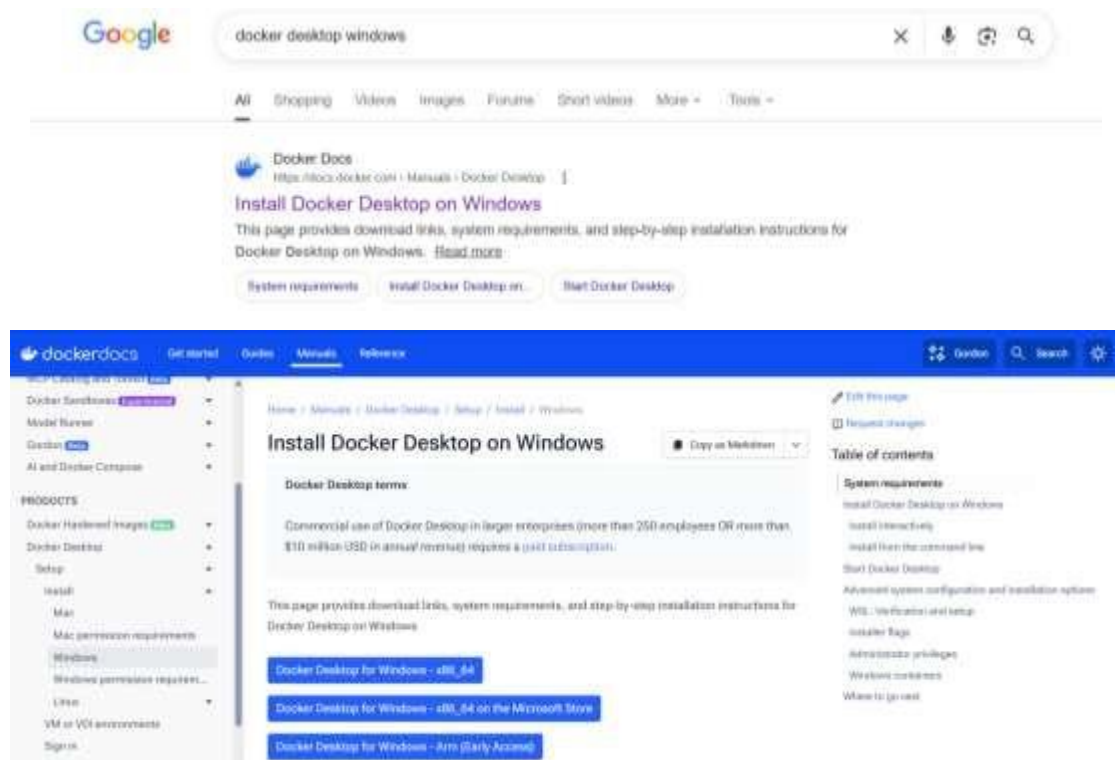
Hyperledger Fabric runs as:

- Docker containers (Containers are like mini virtual machines)

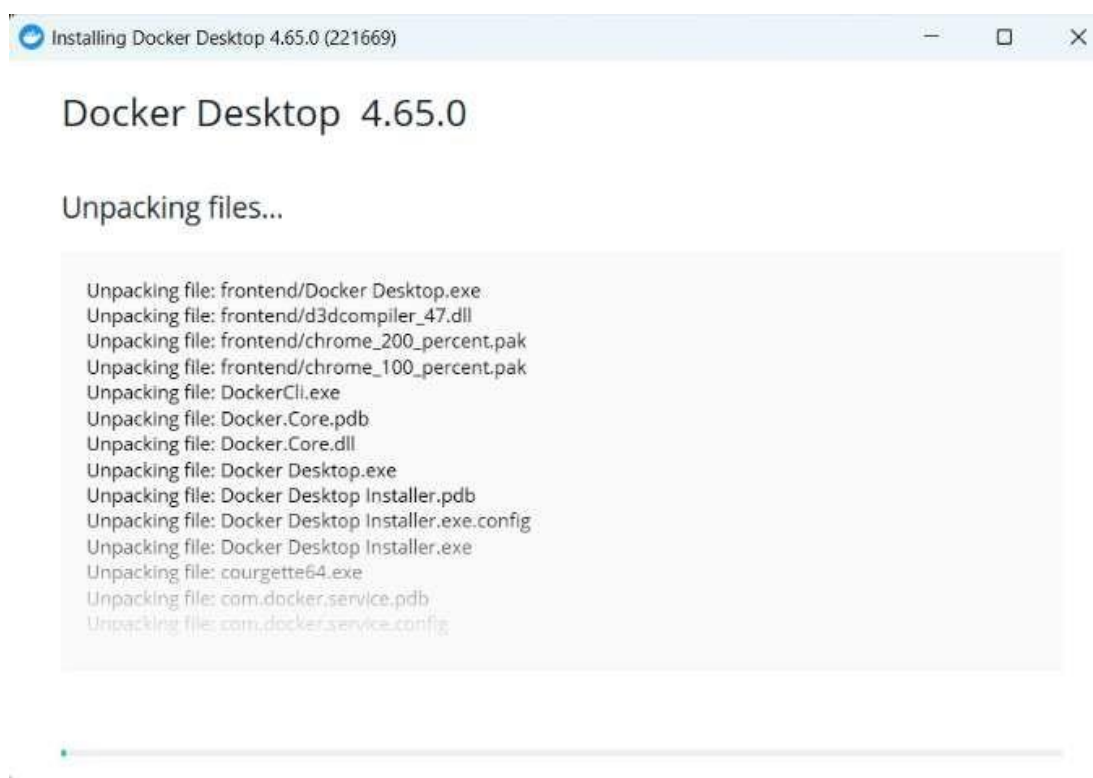
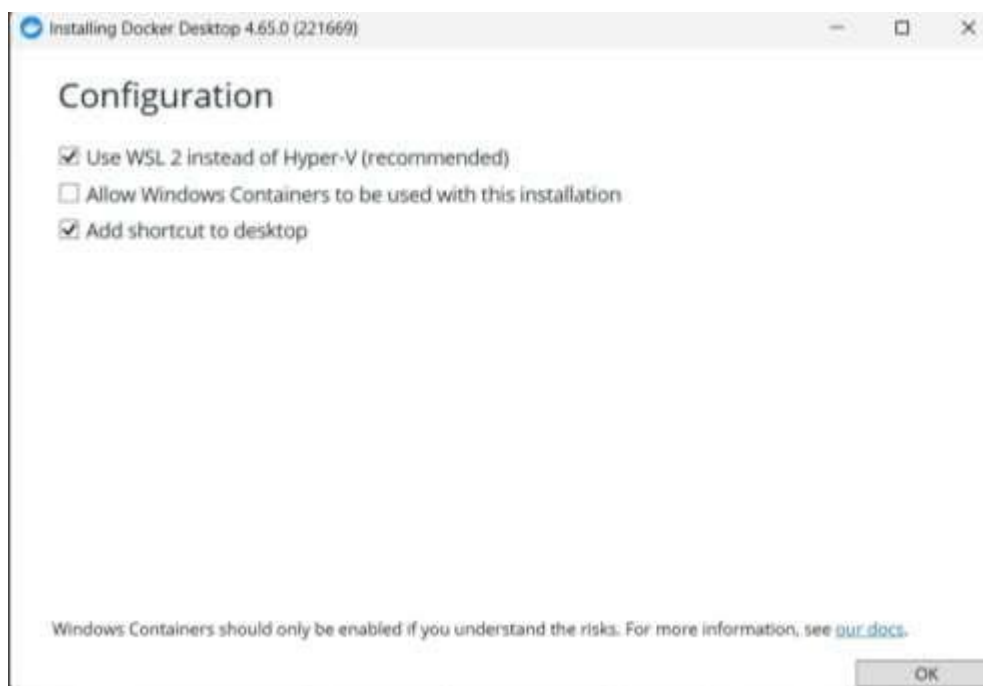
Docker runs:

- Peers
- Orderer
- Certificate authority

A. Install: Open Google Chrome or any other Web Browser you use → Type “**Docker Desktop Windows**” → Click on the link “[Install Docker Desktop on Windows](#)” → Click “[Docker Desktop for Windows - x86_64](#)” → “Docker Desktop Installer.exe” file will be downloaded in the Downloads Folder → Double click the .exe file and perform usual installation.



During installation, if you see something as below, then check in the first checkbox otherwise just check in the “Add shortcut to desktop”, if required and proceed normally with the installation.



NOTE:

- ✓ Enable WSL2 [WSL 2 (Windows Subsystem for Linux 2) is a feature in Windows 10/11 that runs a real Linux kernel in a lightweight virtual machine, allowing users to run GNU/Linux environments directly on Windows without dual-booting.]
- ✓ Enable Linux containers

B. Verify:

docker --version

docker compose version

```
Command Prompt
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shikh>docker --version
Docker version 29.2.1, build a5c7197

C:\Users\shikh>docker compose version
Docker Compose version v5.0.2

C:\Users\shikh>
```

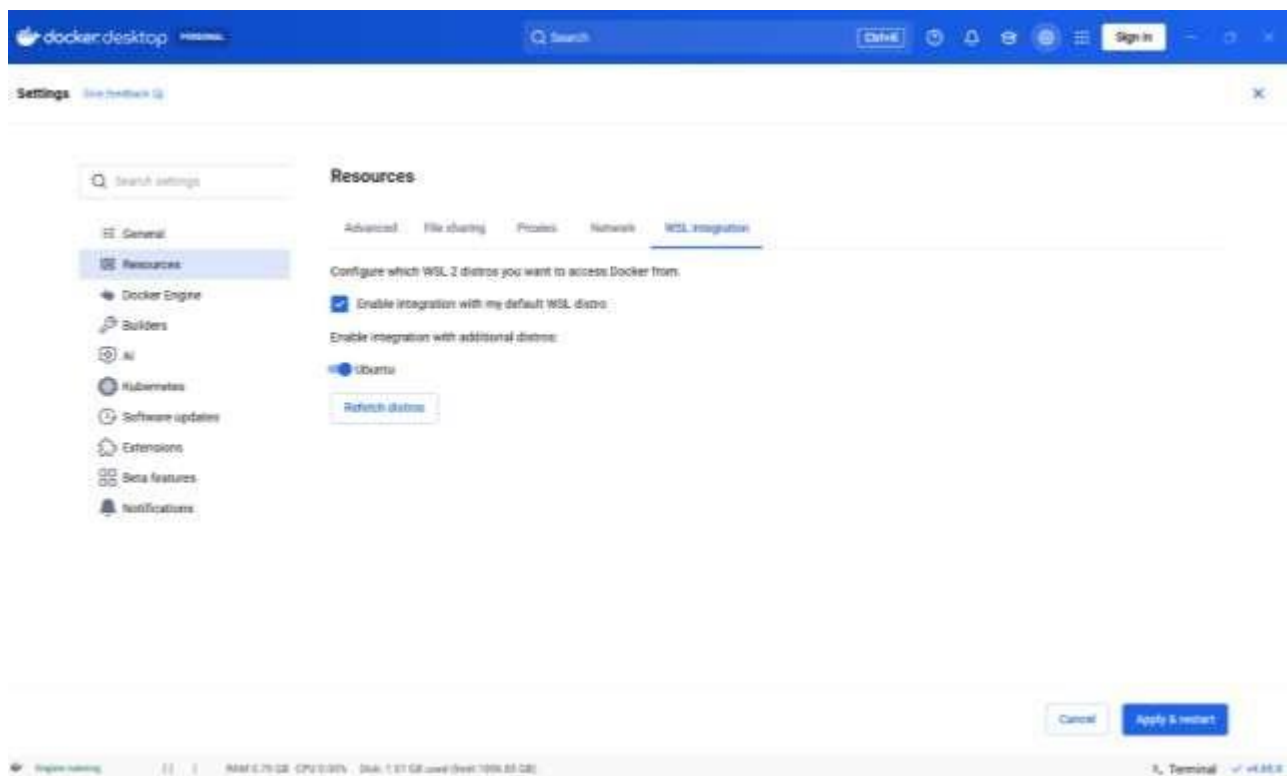
C. # AFTER installation, LAUNCH/OPEN DOCKER DESKTOP, if not launched automatically.

Go to: Settings → Resources → WSL Integration

✓ Check-in the box saying “**Enable integration with my default WSL distro**”

✓ Enable: **Ubuntu**

✓ Click **Apply & restart**



PLEASE ENSURE ONCE INSTALLED, DOCKER DESKTOP MUST BE RUNNING IN ORDER TO PERFORM THIS PRACTICAL

D. Test Docker

Open Ubuntu: **wsl -d Ubuntu**

Run: **docker run hello-world**

Must work before proceeding. You will see something like below:

```
shikh@Shikha: /mnt/c/Users/shikh$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:85404b3c53951c3ff5d40de0972b1bb21fafa2e8daa235355baf44f33db9dbdd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

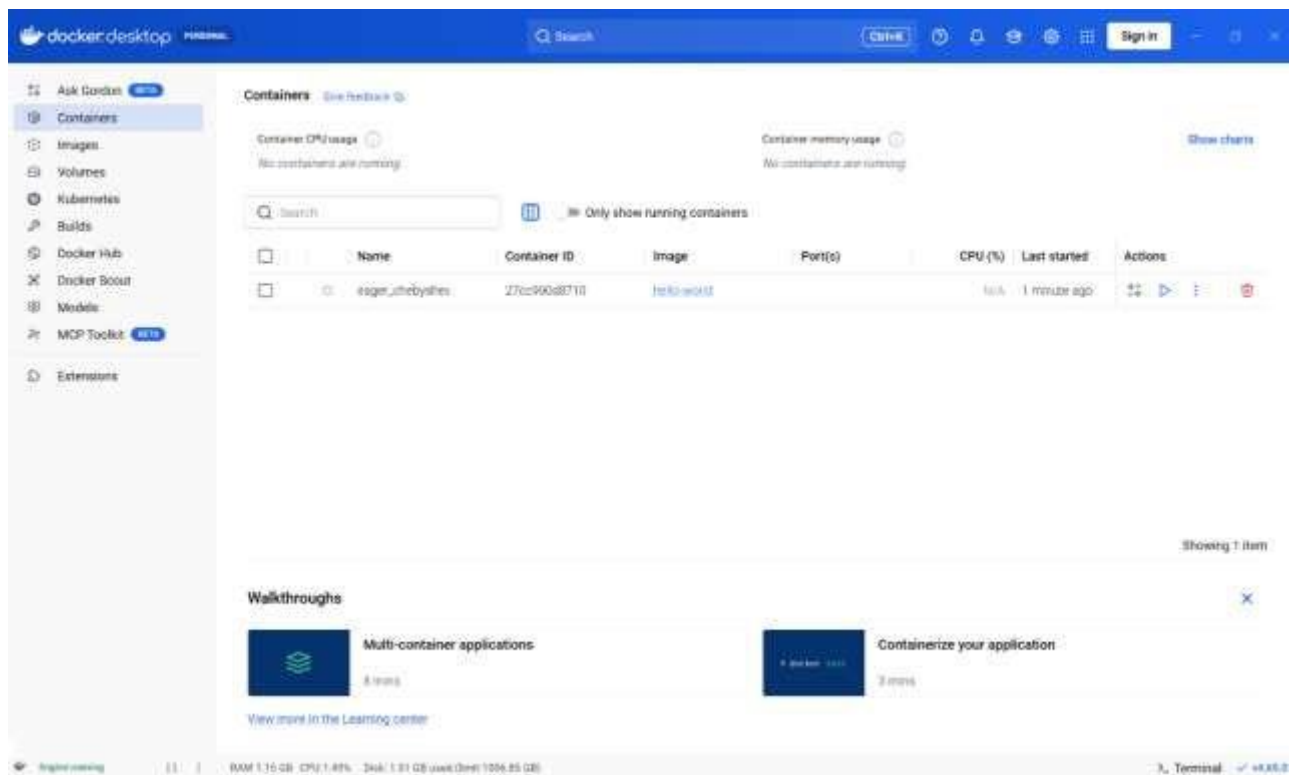
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

shikh@Shikha: /mnt/c/Users/shikh$
```




```

root@osaka:~# sudo apt install -y curl git build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (8.5.0-ubuntu0.8).
curl set to manually installed.
git is already the newest version (1:2.43.0-ubuntu7.3).
git set to manually installed.
The following additional packages will be installed:
  bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu gcc-x86-64-linux-gnu dpkg-dev fakeroot g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13
gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnucxx libalgoritm-diff-perl libalgoritm-diff-xs perl libalgoritm-merge perl libasan libasan4 libatomic-1
libc-dev libc-devtools libc6-dev libc6-i386 libcrypt-dev libc6265-0 libdpkg-perl libfakeroot libfile-fcntllock-perl libgcc-13-dev libgd3 libgpg1 libghcf-plugin-aesenc
libghcf-plugin-aesniwc libghcf-plugin-libse265 libghcf1 libhasan0 libia123 libitall libitsan0 libmpc3 libquasimath0 libstdc++-13-dev libstan2 libubsan1 libubsd1
linux-libc-dev lto-disabled-list make manpages-dev rpcsvc-proto
Suggested packages:
  bzip2-doc cpp-doc gcc-13-locales cpp-13-doc debian-keyring g++-multilib g++-13-multilib gcc-13-doc gcc-multilib autoconf autotools flex bison gdb
gcc-doc gcc-13-multilib gcc-x86-64-linux-gnu glibc-doc brr libgd-tools libghcf-plugin-x265 libghcf-plugin-lua libghcf-plugin-jpegdec libghcf-plugin-jpegenc
libghcf-plugin-jpegnc libghcf-plugin-j2kdec libghcf-plugin-j2kenc libghcf-plugin-ravie libghcf-plugin-vtenc libstdc++-13-doc make-doc
The following NEW packages will be installed:
  build-essential bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu gcc-x86-64-linux-gnu dpkg-dev fakeroot g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu
gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnucxx libalgoritm-diff-perl libalgoritm-diff-xs perl libalgoritm-merge perl libasan4
libasan0 libatomic1 libc-dev bin libc-devtools libc6-dev libc6-i386 libcrypt-dev libc6265-0 libdpkg-perl libfakeroot libfile-fcntllock-perl libgcc-13-dev
libgd3 libgpg1 libghcf-plugin-aesenc libghcf-plugin-aesniwc libghcf-plugin-libse265 libghcf1 libhasan0 libia123 libitall libitsan0 libmpc3 libquasimath0
libstdc++-13-dev libstan2 libubsan1 libubsd1 linux-libc-dev lto-disabled-list make manpages-dev rpcsvc-proto
0 upgraded, 54 newly installed, 0 to remove and 0 not upgraded.
Need to get 71.5 MB of archives.
After this operation, 202 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libc-dev-bin amd64 2.39-0ubuntu8.7 [20.4 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-libc-dev amd64 6.8.0-100.100 [2095 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble/main amd64 libcrypt-dev amd64 2.4.0-1ubuntu1 [122 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 rpcsvc-proto amd64 1.4.2-0ubuntu1 [67.4 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libc6-dev amd64 2.39-0ubuntu8.7 [1210 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 gcc-13-base amd64 13.3.0-6ubuntu2-24.04.1 [51.6 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libia123 amd64 6.36-3build1.1 [686 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libmpc3 amd64 1.3.1-1build1.1 [94.6 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 cpp-13-x86-64-linux-gnu amd64 13.3.0-6ubuntu2-24.04.1 [10.7 MB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 cpp-13 amd64 13.3.0-6ubuntu2-24.04.1 [1002 B]
Get:11 http://archive.ubuntu.com/ubuntu noble/main amd64 gcc-x86-64-linux-gnu amd64 4:13.2.0-ubuntu1 [5326 B]
Get:12 http://archive.ubuntu.com/ubuntu noble/main amd64 cpp amd64 4:13.2.0-ubuntu1 [22.4 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcc1-0 amd64 10.2.0-ubuntu2-20.04.1 [40.0 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libgpg1 amd64 14.2.0-ubuntu2-24.04.1 [118 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libitall amd64 14.2.0-ubuntu2-24.04.1 [29.7 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libstdc++11 amd64 10.2.0-ubuntu2-24.04.1 [110.5 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libasan0 amd64 14.2.0-ubuntu2-24.04.1 [507.0 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libitsan0 amd64 14.2.0-ubuntu2-24.04.1 [1122 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libstan2 amd64 10.2.0-ubuntu2-24.04.1 [2777 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libubsan1 amd64 14.2.0-ubuntu2-24.04.1 [1124 kB]

```

4. Node.js [Node.js is used in Hyperledger Fabric for two primary purposes: to build client applications that interact with the network and as a language for writing **chaincode** (smart contracts). Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to run JavaScript code outside of a web browser, typically for server-side programming. It uses a non-blocking, event-driven architecture, making it highly efficient for scalable, real-time applications. Node.js is integrated into the Hyperledger Fabric ecosystem through its official SDK and as a chaincode language, offering flexibility and accessibility to a wide range of developers already proficient in JavaScript.]

Why needed? → Fabric CLI tools and scripts depend on Node runtime.

Install: Stay in the same terminal location, i.e., “shikh@Shikha:~\$” for further installation.

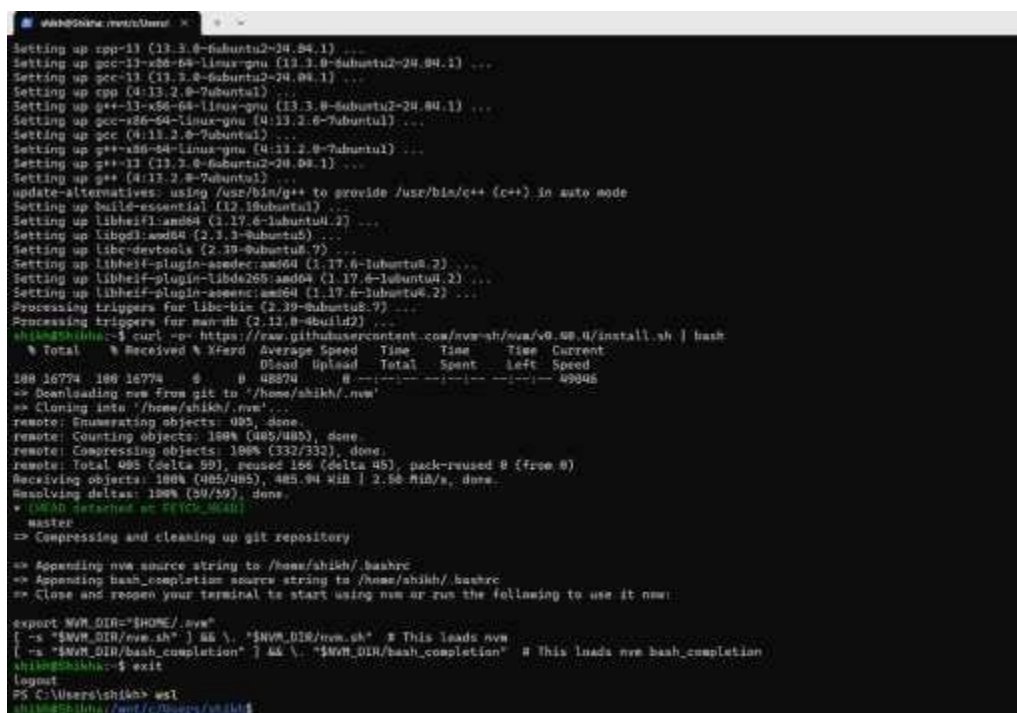
a. Go to: <https://nodejs.org/> → Click **Get Node.js®** → Select Get Node.js® **v24.14.0** (LTS) for **Linux** using **nvm** with **npm**



b. Copy and run the following command given on the above link and shown in above screenshot:

curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash

Once it finishes, you **must reload the terminal**. First, type: **exit**, then reopen Ubuntu: **ws!**



c. Then install Node.js

After reload, run the following commands:

```
nvm install 24
```

```
nvm use 24
```

```
export NVN_DIR="$HOME/.nvm"
[ -s "$NVN_DIR/nvm.sh" ] && \. "$NVN_DIR/nvm.sh" # This loads nvm
[ -s "$NVN_DIR/bash_completion" ] && \. "$NVN_DIR/bash_completion" # This loads nvm bash_completion
shikh@Shikha:~$ exit
logout
PS C:\Users\shikh> wsl
shikh@Shikha:/mnt/c/Users/shikh$ nvm install 24
Downloading and installing node v24.14.0...
Downloading https://nodejs.org/dist/v24.14.0/node-v24.14.0-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v24.14.0 (npm v11.9.0)
Creating default alias: default -> 24 (-> v24.14.0 *)
```

Check:

```
node -v
```

```
npm -v
```

```
shikh@Shikha:/mnt/c/Users/shikh$ node -v
v24.14.0
shikh@Shikha:/mnt/c/Users/shikh$ nvm -v
0.40.4
shikh@Shikha:/mnt/c/Users/shikh$
```

5. cURL (Client URL) (Curl is generally preinstalled in recent versions of Windows):

- It is used to transfer data to and from a server, often utilized to interact with blockchain nodes, API endpoints, and smart contracts.
- Purpose: It allows developers to send HTTP/HTTPS requests (GET, POST) directly from the command line to communicate with blockchain APIs (e.g., Ethereum JSON-RPC, Hyperledger Fabric, Bitcoin Core).
- Functionality: It is frequently used to query data from a blockchain, submit transactions, or check node status.
- Pronunciation: Pronounced "see URL".
- Components:
 - curl: The command-line tool.
 - libcurl: The library that supports the data transfer protocols.

Why needed?

To download Fabric binaries using command line.

Install: <https://curl.se/windows/>

Verify: **curl --version**

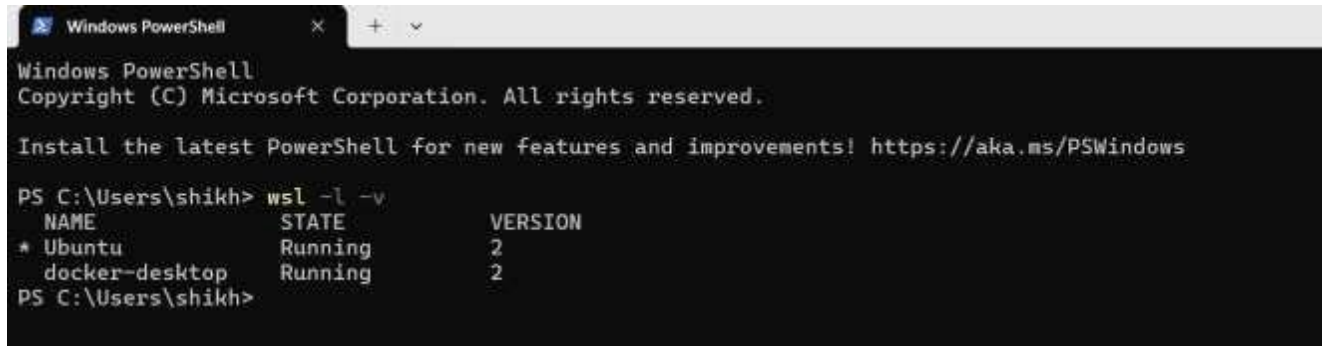
```
Microsoft Windows [Version 10.0.26200.7848]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shikh>curl --version
curl 8.16.0 (Windows) libcurl/8.16.0 Schannel zlib/1.3.1 WinIDN
Release-Date: 2025-09-18
Protocols: dict file ftp ftps gopher gophers http https imap imaps ipfs ipns ldap ldaps mqtt pop3 pop3s smb smbs smtp satps telnet tftp ws wss
Features: alt-svc AsynchDNS HSTS HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM SPNEGO SSL SSPI threadsafe Unicode UnixSockets

C:\Users\shikh>
```

III. IMPLEMENTATION

NOTE: ENSURE BOTH WSL AND DOCKER DESKTOP IS RUNNING before performing further steps. You can do so by opening a new Terminal Window and running the following command: **"wsl -l -v"**. If you see something as below, you are good to go.



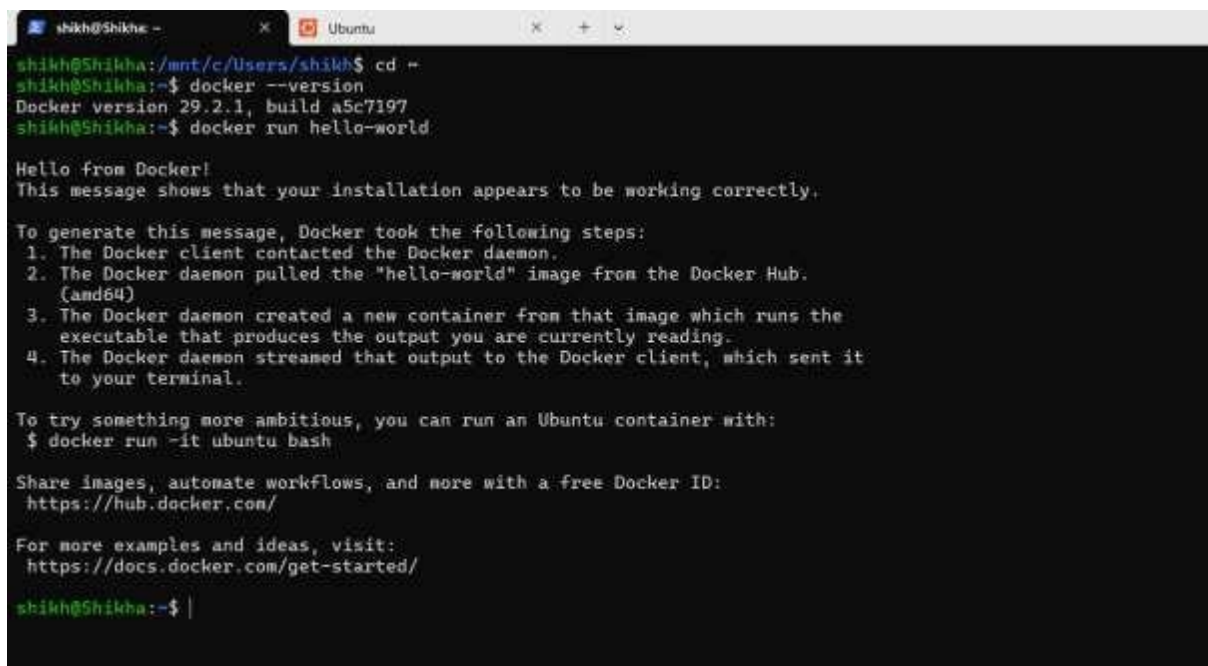
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shikh> wsl -l -v
  NAME                STATE          VERSION
  * Ubuntu              Running         2
  docker-desktop        Running         2
PS C:\Users\shikh>
```

Further, to run/perform following steps, either **open Ubuntu** by clicking the drop down adjacent to the already running Terminal Window where we have performed all the installations **OR** change the directory of the same Terminal Window to your home directory by running **"cd ~"** command which will give you the following prompt: **"shikh@Shikha:~\$"**.

You can first test integration of docker again by running the following command: **docker --version** and then **docker run hello-world**.



```
shikh@Shikha:~$ cd ~
shikh@Shikha:~$ docker --version
Docker version 20.10.11, build a5c7197
shikh@Shikha:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

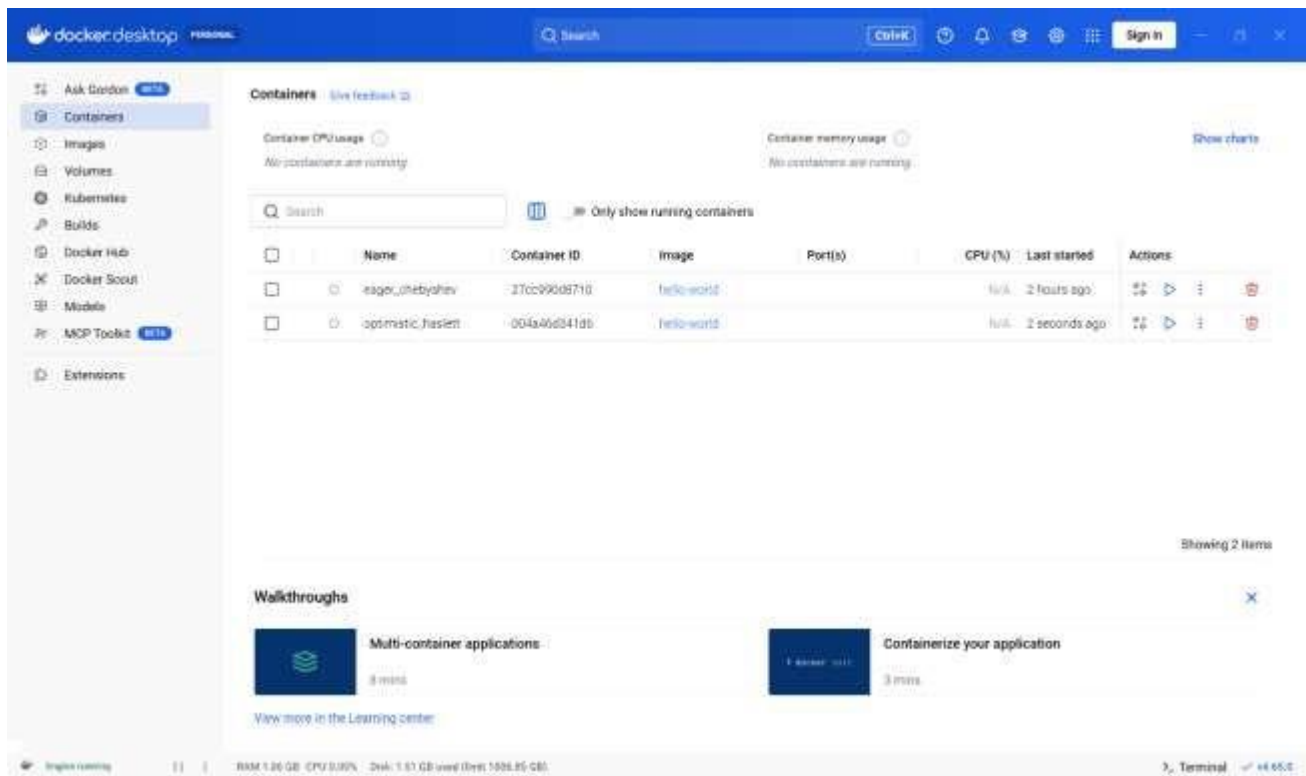
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

shikh@Shikha:~$ |
```



STEP 1: Download Fabric Samples

Run: **`curl -sSL https://bit.ly/2ysbOFE | bash -s`**

This step downloads and runs the official Hyperledger Fabric installation script. Specifically, it:

- ✓ Downloads Fabric binaries
- ✓ Downloads Docker images
- ✓ Downloads sample networks
- ✓ Set up test-network
- ✓ Creates fabric-samples folder

Flag/Code	Meaning
-s	Silent mode (no progress bar); download silently
-S	Show errors if something fails
-L	Follow redirected link
https://bit.ly/2ysbOFE	A shortened link; redirects to official Hyperledger Fabric bootstrap (shell) script on GitHub maintained by the Hyperledger team
	Take output of left command and give it as input to right command; curl downloads script; passes script to bash
bash -s	bash is Linux shell interpreter; -s means read commands from standard input; so, the downloaded script is executed directly by bash.

When you run this:

1. Script downloads: Fabric binaries, Docker images, fabric-samples repository
2. It creates: fabric-samples/
3. Inside it:
 - test-network/

- bin/
 - config/
4. Docker images are pulled like:
- peer image
 - orderer image
 - tools image

```

shah@Shaha:~$ curl -sL https://bit.ly/2ystdFE | bash -s
Clone hyperledger/fabric-samples repo
====> Cloning hyperledger/fabric-samples repo
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15591, done.
remote: Counting objects: 100% (154/154), done.
remote: Compressing objects: 100% (59/59), done.
remote: Total 15591 (delta 148), reused 95 (delta 93), pack-reused 15591 (from 15)
Receiving objects: 100% (15591/15591), 26.86 MiB | 3.64 MiB/s, done.
Resolving deltas: 100% (3792/3792), done.
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries
====> Downloading version 2.5.15 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
% Total % Received % Xferd Average Speed Time Time Current
Dload Upload Total Speed Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100 120M 100 120M 0 0 11.5M 0 0:00:10 0:00:10 12.4M
====> Done.
====> Downloading version 1.5.17 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
% Total % Received % Xferd Average Speed Time Time Current
Dload Upload Total Speed Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
100 31.0M 100 31.0M 0 0 6651k 0 0:00:04 0:00:04 6650k
====> Done.

Pull Hyperledger Fabric docker images
FABRIC_IMAGES: peer orderer cconv basess
====> Pulling fabric images
====> docker.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
6226c6743f4: Pull complete
abf36fcd3031: Pull complete
5f3ba2a042ca: Pull complete
1483ce11320c: Pull complete
8a2ab97a7a1c: Pull complete
3d33773e2b04: Pull complete
8df2aae32617: Download complete
Digest: sha256:c48323771397cabe77a98c44eeaf46d627dec225f754866f52422393a8bd59
Status: Downloaded newer image for hyperledger/fabric-peer:2.5.15
====> docker.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
229a38ab26d4: Pull complete
94c191cca972: Pull complete
864da70cedcc: Pull complete
18b1879d5dab: Pull complete
864ff62b2b7a: Pull complete
433dc9a1c517: Download complete
Digest: sha256:72647d999f45c6e37f386d8a09fbed861c1d3883c7b28ec6169577d8413b791

```

```

18b1879d5dab: Pull complete
864ff62b2b7a: Pull complete
433dc9a1c517: Download complete
Digest: sha256:72647d999f45c6e37f386d8a09fbed861c1d3883c7b28ec6169577d8413b791
Status: Downloaded newer image for hyperledger/fabric-orderer:2.5.15
====> docker.io/hyperledger/fabric-cconv:2.5.15
2.5.15: Pulling from hyperledger/fabric-cconv
40462a118762: Pull complete
b44dc2336b4: Pull complete
c26c44090b6: Pull complete
6c36f8a9c981: Pull complete
f18ec371f6c2: Pull complete
2b6b6d8c8c4: Pull complete
ee1459a05689: Download complete
Digest: sha256:20cra483a20845f8e7crafc35868e02c8a9c13e1768f7e881e7b9f326f3a5e7
Status: Downloaded newer image for hyperledger/fabric-cconv:2.5.15
====> docker.io/hyperledger/fabric-basess:2.5.15
2.5.15: Pulling from hyperledger/fabric-basess
7ad9b5374d96: Pull complete
027f6e5c50f: Pull complete
5cbf1f728a88: Pull complete
07ab3c77c2e5: Download complete
Digest: sha256:c8b3d4193d7a221c7783b07879423d7d562d36c4b66e1675f229f2a721d7f15
Status: Downloaded newer image for hyperledger/fabric-basess:2.5.15
====> Pulling fabric ca image
====> docker.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
c474c9d8dc: Pull complete
644ebca3823: Pull complete
9c137a7ef4f: Pull complete
10640d3f854a: Pull complete
81e770ba1377: Download complete
Digest: sha256:fac8ca78a7c346aa7017fbbcd276b77ac336cf5b5410b41c7363586e56a
Status: Downloaded newer image for hyperledger/fabric-ca:1.5.17
====> docker.io/hyperledger/fabric-ca:1.5.17
====> List out hyperledger docker images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
hyperledger/fabric-basess:2.5 c8b3d4193d7a 25MB 88.4MB
hyperledger/fabric-basess:2.5.15 c8b3d4193d7a 25MB 88.4MB
hyperledger/fabric-basess:latest c8b3d4193d7a 25MB 88.4MB
hyperledger/fabric-ca:1.5 4ac8ca78a7c 381MB 127MB
hyperledger/fabric-ca:1.5.17 4ac8ca78a7c 381MB 127MB
hyperledger/fabric-ca:latest 4ac8ca78a7c 381MB 127MB
hyperledger/fabric-cconv:2.5 20cra483a269 1GB 235MB
hyperledger/fabric-cconv:2.5.15 20cra483a269 1GB 235MB
hyperledger/fabric-cconv:latest 20cra483a269 1GB 235MB
hyperledger/fabric-orderer:2.5 72647d999f45 17MB 40.5MB
hyperledger/fabric-orderer:2.5.15 72647d999f45 17MB 40.5MB
hyperledger/fabric-orderer:latest 72647d999f45 17MB 40.5MB
hyperledger/fabric-peer:2.5 c4832377139 23MB 66.2MB
hyperledger/fabric-peer:2.5.15 c4832377139 23MB 66.2MB
hyperledger/fabric-peer:latest c4832377139 23MB 66.2MB
shah@Shaha:~$

```

STEP 2: Go to Test Network

cd fabric-samples/test-network

Test-network is:

- Pre-configured mini blockchain
- Created for learning

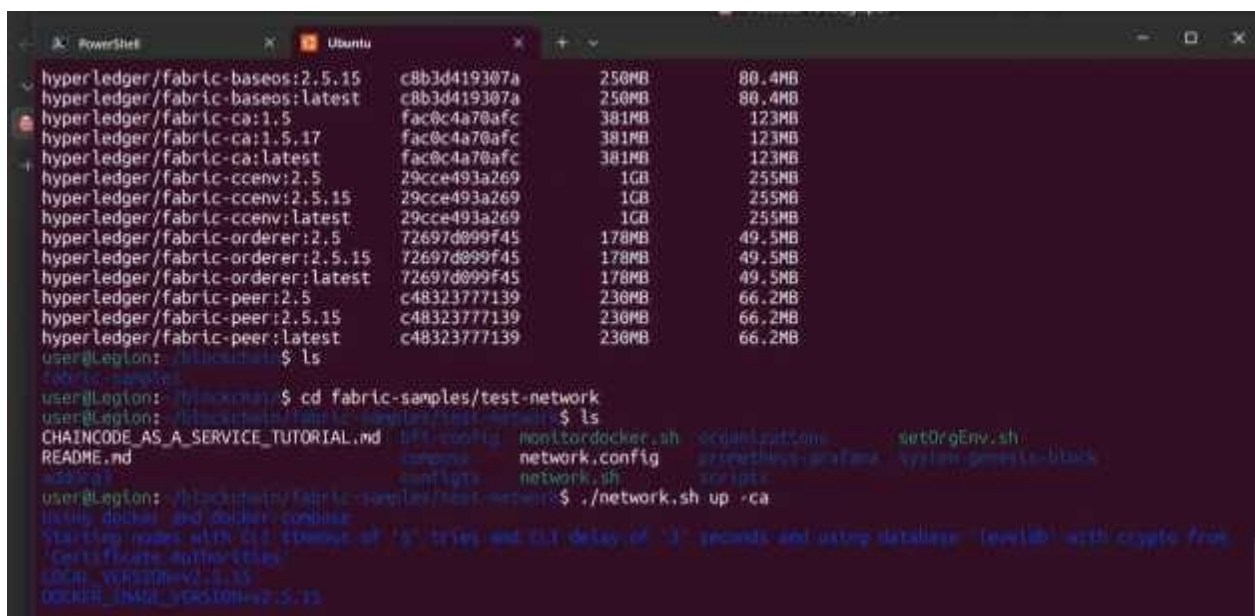
STEP 3: Start Blockchain Network

./network.sh up -ca

Here, Docker creates containers for:

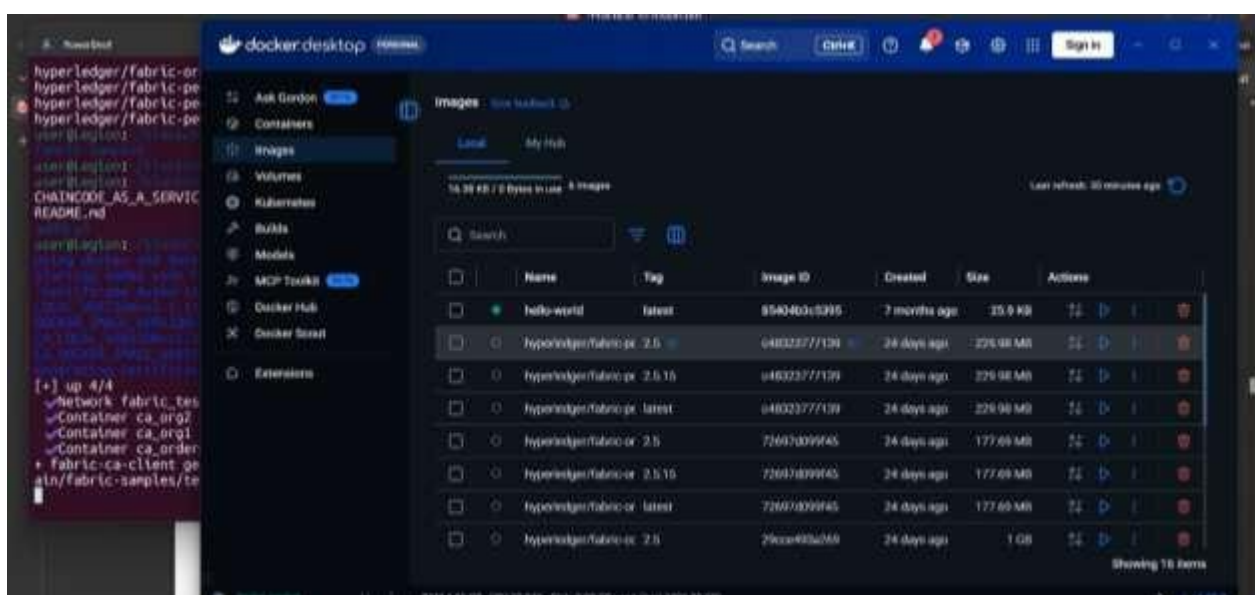
- Peer0 Org1
- Peer0 Org2
- Orderer
- Certificate authority

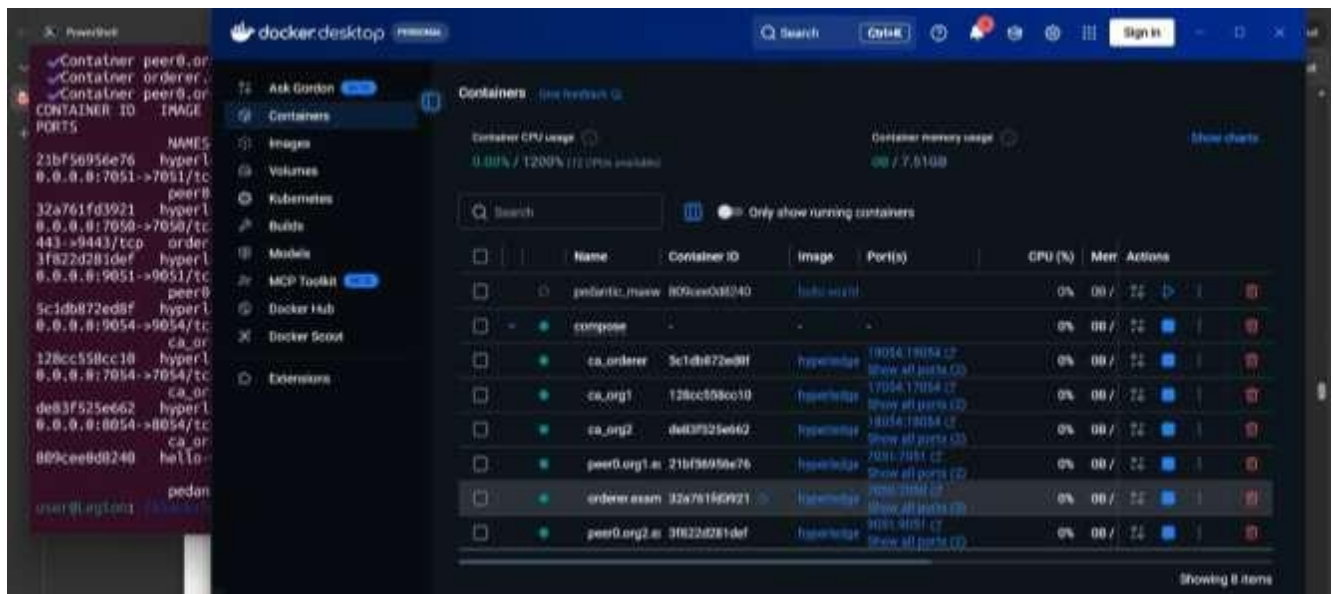
After completion: Network is LIVE.



```
hyperledger/fabric-baseos:2.5.15    c8b3d419307a    250MB    80.4MB
hyperledger/fabric-baseos:latest    c8b3d419307a    250MB    80.4MB
hyperledger/fabric-ca:1.5           fac0c4a70afc    381MB    123MB
hyperledger/fabric-ca:1.5.17        fac0c4a70afc    381MB    123MB
hyperledger/fabric-ca:latest         fac0c4a70afc    381MB    123MB
hyperledger/fabric-ccenv:2.5         29cce493a269    1GB      255MB
hyperledger/fabric-ccenv:2.5.15      29cce493a269    1GB      255MB
hyperledger/fabric-ccenv:latest      29cce493a269    1GB      255MB
hyperledger/fabric-orderer:2.5       72697d099f45    178MB    49.5MB
hyperledger/fabric-orderer:2.5.15    72697d099f45    178MB    49.5MB
hyperledger/fabric-orderer:latest    72697d099f45    178MB    49.5MB
hyperledger/fabric-peer:2.5          c48323777139    230MB    66.2MB
hyperledger/fabric-peer:2.5.15       c48323777139    230MB    66.2MB
hyperledger/fabric-peer:latest       c48323777139    230MB    66.2MB

user@legion: /blockchain $ ls
fabric-samples
user@legion: /blockchain $ cd fabric-samples/test-network
user@legion: /blockchain/fabric-samples/test-network $ ls
CHAINCODE_AS_A_SERVICE_TUTORIAL.md  cli-config  monitorDocker.sh  organizations  setOrgEnv.sh
README.md                             compose     network.config    arweave-grafana  system-generators
addPeer.sh                           configs     network.sh        scripts
user@legion: /blockchain/fabric-samples/test-network $ ./network.sh up -ca
Using docker and docker compose
Starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb' with crypto from
'Certificate Authorities'
LOCAL_VERSION=2.5.15
DOCKER_IMAGE_VERSION=2.5.15
```





Explanation: *network.sh* is a file inside *fabric-samples/test-network/*. (It is a shell script.)

Flag/Code	Meaning
<i>network.sh</i>	a file inside <i>fabric-samples/test-network/</i> ; it is a shell script
<i>./</i>	run the script located in the current directory
up	up → Start the network down → Stop the network createChannel → Create channel deployCC → Deploy chaincode
<i>./network.sh up</i>	Start the Hyperledger Fabric test network.
-ca	Certificate Authority

When you run this:

The script uses **Docker** to create containers. It starts the following components:

1. Certificate Authorities (CA): Fabric is permissioned, so identity is required. CA:

- Issues digital certificates
- Authenticates users and nodes

Without CA:

- No trusted identity
- No permissioned system

2. Peers: It creates:

- peer0.org1.example.com
- peer0.org2.example.com

Each peer:

- Stores blockchain ledger
- Executes chaincode
- Validates transactions

Think of peer as a blockchain participant computer.

3. **Orderer**: Creates orderer.example.com. Orderer:

- Collects transactions
- Orders them into blocks
- Distributes blocks to peers

No mining is involved.

4. **Genesis Block Creation**: The first block of the channel is created. Genesis block contains:

- Network configuration
- Member organizations
- Orderer details

What You See on Screen: When running: `./network.sh up`; you will see:

- Docker images pulling
- Containers starting
- Logs of peer and orderer

At end, you will see something like: Network is up

That means:

- ✓ Fabric network is running
- ✓ Blockchain components are live

STEP 4: Create Channel

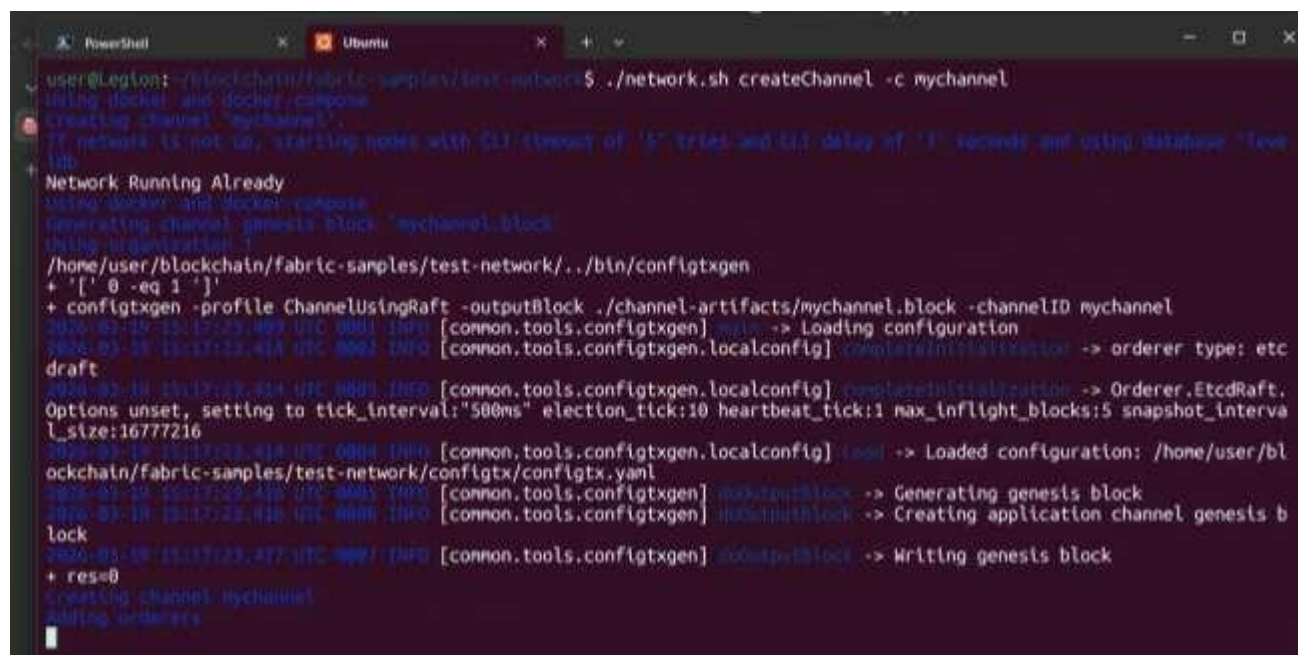
`./network.sh createChannel -c mychannel`

[Here, `-c` stands for **channel** and ***mychannel*** is the **name of the channel** you are creating]

What this does:

- Creates channel named mychannel
- Connects peers to that channel

Now organizations can transact privately.



```
user@legion: /blockchain/fabric-samples/test-network$ ./network.sh createChannel -c mychannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with $(1) timeout of '5' tries and $(2) delay of '1' seconds and using database 'leveldb'
Network Running Already
Using docker and docker-compose
Generating channel genesis block 'mychannel.block'
Using organization 1
/home/user/blockchain/fabric-samples/test-network/./bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/mychannel.block -channelID mychannel
2020-03-10 15:17:23.409 UTC [INFO] [common.tools.configtxgen] main -> Loading configuration
2020-03-10 15:17:23.414 UTC [INFO] [common.tools.configtxgen.localconfig] completeInitialization -> orderer type: etc draft
2020-03-10 15:17:23.414 UTC [INFO] [common.tools.configtxgen.localconfig] completeInitialization -> Orderer.EtcdRaft.
Options unset, setting to tick_interval:"500ms" election_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2020-03-10 15:17:23.414 UTC [INFO] [common.tools.configtxgen.localconfig] load -> Loaded configuration: /home/user/blockchain/fabric-samples/test-network/configtx/configtx.yaml
2020-03-10 15:17:23.414 UTC [INFO] [common.tools.configtxgen] maxOutputBlock -> Generating genesis block
2020-03-10 15:17:23.414 UTC [INFO] [common.tools.configtxgen] maxOutputBlock -> Creating application channel genesis block
2020-03-10 15:17:23.417 UTC [INFO] [common.tools.configtxgen] maxOutputBlock -> Writing genesis block
+ res=0
Creating channel 'mychannel'
Adding orderers
```



```
PowerShell
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label
basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
```

While running this command, you may get **ERROR** saying:

“Chaincode installation on peer0.org1 failed”

“Deploying chaincode failed”

Then you should first run following command and then re-run above command to deploy chaincode. The command is: **docker pull Hyperledger/fabric-nodeenv:2.5**

```
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
+ res=1
Error: chaincode install failed with status: 500 - failed to invoke backing implementation of 'InstallChaincode': could
not build chaincode: docker build failed: docker image build failed: error during connect: Post "http://%2Fhost%2Fvar%2F
run%2Fdocker.sock/v1.52/build?networkmode=fabric_test&pull=1&rm=0&t=dev-peer0.org1.example.com-basic_1.0-1d4d44557311281
3889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa885d60dc": doc
ker build failed: Error creating container: Error response from daemon: No such image: hyperledger/fabric-nodeenv:2.5
Chaincode installation on peer0.org1 has failed
Deploying chaincode failed
user@legion: ~/blockchain/fabric-samples/test-network$ docker pull hyperledger/fabric-nodeenv:2.5
2.5: Pulling from hyperledger/fabric-nodeenv
015820c03a84: Pulling fs layer
2bea4be8c590: Pulling fs layer
89715c6acb26: Pulling fs layer
a1c7bd30f9ab: Pulling fs layer
c3f44fc696cb: Pulling fs layer
03d2f4babaac: Pulling fs layer
Digest: sha256:17e2d447ca0de5b4e3f6950a1c9b24ecfdeecdd90e111e11d771970d35159bf1
Status: Downloaded newer image for hyperledger/fabric-nodeenv:2.5
docker.io/hyperledger/fabric-nodeenv:2.5
user@legion: ~/blockchain/fabric-samples/test-network$
```

Explanation:

Option	Meaning
deployCC	Deploy ChainCode; Installs and activates the smart contract on the blockchain network; Internally it performs multiple steps automatically:

Option	Meaning
	1. Package chaincode 2. Install chaincode on peers 3. Approve chaincode definition 4. Commit chaincode to channel
-ccn basic	Chaincode name; name of the smart contract is "basic"
-ccp	Chaincode path (Where is the smart contract code located?); Path meaning: • .. → go one folder back • asset-transfer-basic → sample contract folder • chaincode-go → Go language version So this points to: A sample smart contract that manages assets.
-ccl javascript	Chaincode Language; Fabric supports: Go, Java, JavaScript (Node.js); here, Smart contract is written in "JavaScript" language.

This deploys Asset transfer smart contract.

STEP 6: Set Environment Variables, Org1 environment and other relevant exports

```
export PATH=${PWD}/../bin:$PATH
```

```
export FABRIC_CFG_PATH=$PWD/../config/
```

This tells the system where Fabric binaries are located.

Option	Meaning
LINE 1 (export PATH=\${PWD}/../bin:\$PATH)	
Export	export means create or modify an environment variable for the current terminal session. Environment variables are special variables that tell programs: where files are located, how to behave, which configuration to use
PATH	PATH is a system variable that tells Linux where to search for executable programs. When we type: "peer"; Linux searches in folders listed in PATH.
\${PWD}	PWD means Present Working Directory (current folder). If we are inside: fabric-samples/test-network; then, \${PWD} is: fabric-samples/test-network
../bin	.. means go one folder up. So: \${PWD}/../bin means: fabric-samples/bin That folder contains: peer binary, orderer binary, configtxgen, cryptogen. These are Fabric CLI tools.
:\$PATH	It appends the existing PATH. So we are saying: Add Fabric's bin folder to the system PATH without removing existing PATH.
Final Meaning	This line tells the system: "When I type Fabric commands like peer, look inside the Fabric bin folder." Without this, the system will say: peer: command not found; Adds Fabric command tools to the system so we can use peer and orderer commands.
LINE 2 (export FABRIC_CFG_PATH=\$PWD/../config/)	
FABRIC_CFG_PATH	This is a Fabric-specific variable. It tells Fabric where to find configuration files. These define: Peer settings, Orderer settings, Network parameters
\$PWD/../config/	If you are inside: fabric-samples/test-network, then: \$PWD/../config/ means: fabric-samples/config. That folder contains: core.yaml

Option	Meaning
	When you run: "peer chaincode query"; the peer command needs: Network configuration, MSP settings, Logging settings. Without setting this variable, you may get errors like: Cannot find core.yaml
Final Meaning	Tells Fabric where its configuration files are located.

These commands are needed because when using CLI manually (instead of network.sh script):

- Fabric binaries must be accessible
- Fabric config files must be accessible

These two lines prepare your terminal to work with Fabric.

```
export CORE_PEER_TLS_ENABLED=true
```

```
export CORE_PEER_LOCALMSPID="Org1MSP"
```

```
export
```

```
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
```

```
export
```

```
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
```

```
export CORE_PEER_ADDRESS=localhost:7051
```

#Some exports to use invoke command

```
export
```

```
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
export
```

```
ORG1_CA=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
```

```
export
```

```
ORG2_CA=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

```

user@Legion: ~/Blockchain/Fabric-101-samples/test-network$ export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
user@Legion: ~/Blockchain/Fabric-101-samples/test-network$ export ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
export ORG1_CA=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export ORG2_CA=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
user@Legion: ~/Blockchain/Fabric-101-samples/test-network$

```

STEP 7: Query Ledger (Read Data) (Show me all assets stored in the ledger.) [This is a read-only operation. No new block is created. No transaction is committed.]

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

Meaning:

- **peer** (CLI tool of Hyperledger Fabric) → call peer node (It allows you to: Query ledger, Invoke smart contract, Join channel, Install chaincode)
- **chaincode** → execute smart contract
- **query** → read-only operation (It does NOT: Create a transaction, Create a block, Change ledger. It only fetches data.)
- **-C mychannel** → -C means channel and *mychannel* is the channel name (Channel = private communication layer between organizations. So this means: Query the smart contract deployed on channel "mychannel".)
- **-n basic** → chaincode name
- **-c '{"Args":["GetAllAssets"]}'** → This is the most important part. This JSON tells the smart contract: Call function "GetAllAssets". Let's understand it.

"Args" → Arguments passed to the chaincode function.

Here, ["GetAllAssets"] means: Call function named GetAllAssets with no parameters.

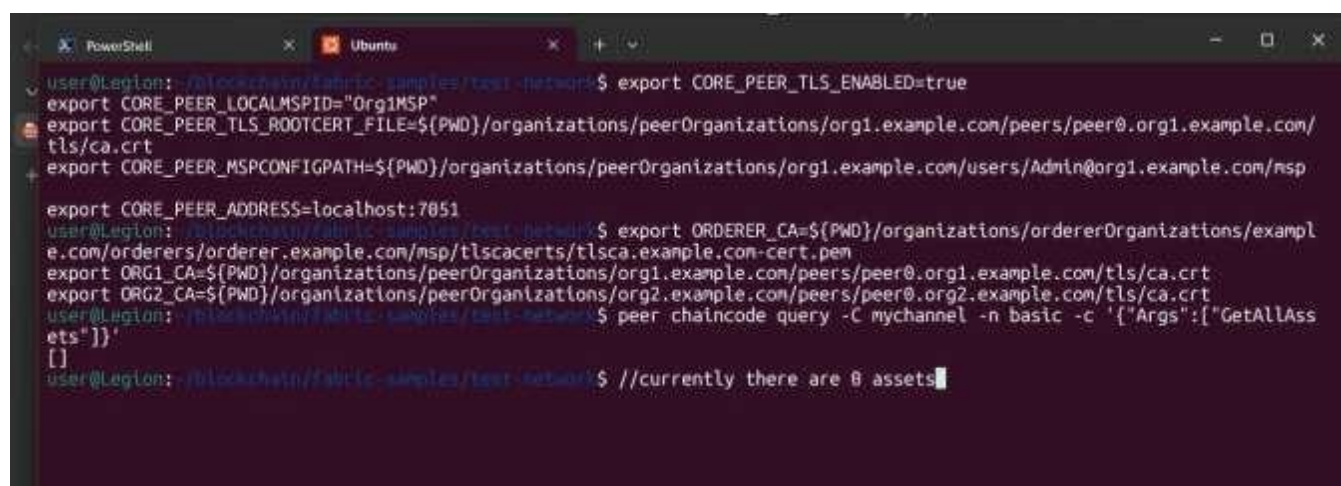
What Is GetAllAssets?

In the sample asset-transfer chaincode, GetAllAssets() reads all asset entries and returns them in JSON format. Each asset looks like:

```
{
  "ID": "asset1",
  "Color": "blue",
  "Size": 5,
  "Owner": "Tom",
  "Value": 300
}
```

Output: List of assets stored in ledger.

Currently, if you try to query ledger, you will not get anything as there are 0 assets as of now. You will receive an empty array/list like [].



```
user@Legion: /blockchain/fabric-samples/test-network$ export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$(PWD)/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$(PWD)/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
user@Legion: /blockchain/fabric-samples/test-network$ export ORDERER_CA=$(PWD)/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
export ORG1_CA=$(PWD)/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export ORG2_CA=$(PWD)/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
user@Legion: /blockchain/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[]
user@Legion: /blockchain/fabric-samples/test-network$ //currently there are 0 assets
```

STEP 8: Invoke Transaction (Write Data) (This creates a new asset and writes it to the blockchain.)

We are telling the blockchain: "Create a new asset with ID asset7 and store it permanently in the ledger."

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_CA --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_CA -c '{"function":"CreateAsset","Args":["asset7","blue","50","student","300"]}'
```

invoke means: Execute a function that modifies the ledger. Unlike query, this Creates transaction proposal, Goes to orderer, Creates block, Updates ledger

-c '{"Args":["...]}': This JSON defines: Function name, Parameters. This means, Call function: CreateAsset(ID, Color, Size, Owner, Value). So we are creating:

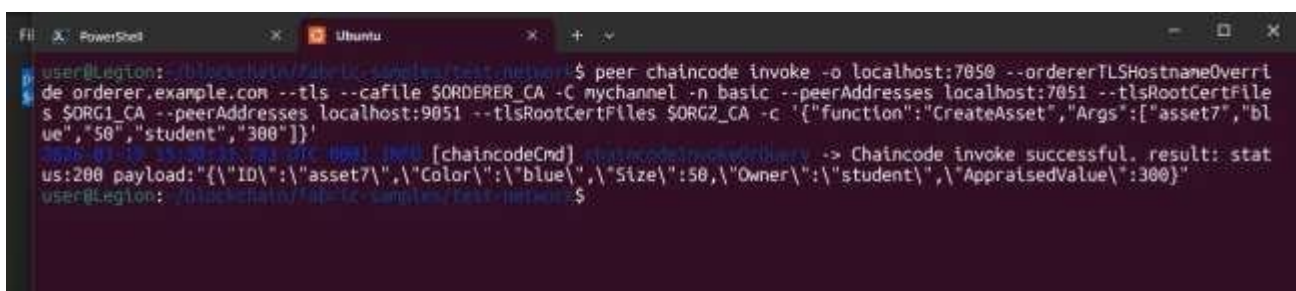
Field	Value
ID	asset7
Color	blue
Size	50
Owner	student
Value	300

Transaction goes through: Client → Peer (endorsement) → Orderer → Block created → Peers validate → Ledger updated

Output: We may see something like: Chaincode invoke successful. result: status:200

It may also show: transaction ID: xxxxx

That transaction ID uniquely identifies our transaction.



```
user@Legion: /blockchain/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_CA --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_CA -c '{"function":"CreateAsset","Args":["asset7","blue","50","student","300"]}'
2020-08-10 15:00:13.843 UTC [chaincodeCmd] ChaincodeInvokeStub-> Chaincode invoke successful. result: status:200 payload:{"ID":"asset7","Color":"blue","Size":50,"Owner":"student","AppraisedValue":300}
user@Legion: /blockchain/fabric-samples/test-network$
```

STEP 9: Verify Transaction

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset7"]}'
```

OR

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

Now we are performing a **read operation for a specific asset** from the blockchain. We will see newly created asset. Blockchain working successfully. This JSON means, Call function: ReadAsset(assetID) With parameter: asset7

What Is ReadAsset?

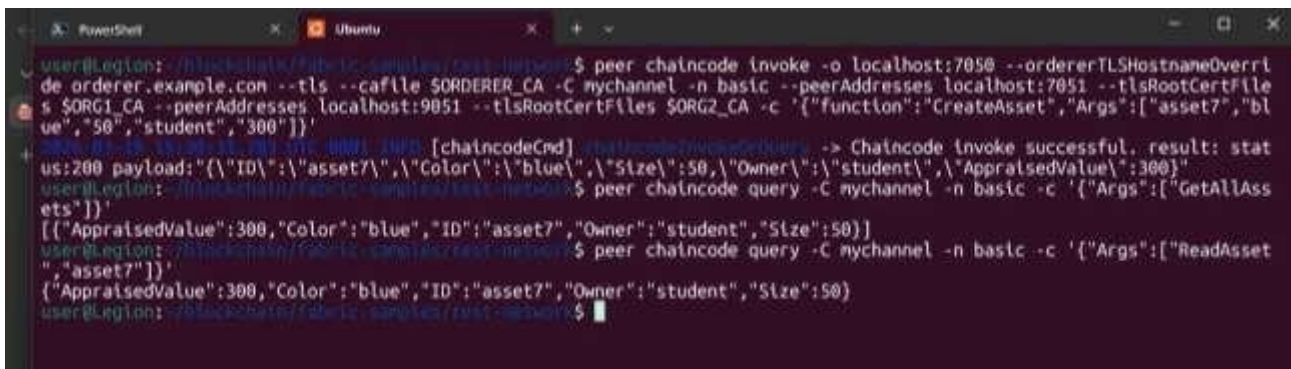
Inside the asset-transfer chaincode, the function: ReadAsset(ID) Looks up asset in world state, Returns asset data in JSON format

Expected Output: If asset7 exists (which we created earlier), we should see something like:

```
{
  "ID": "asset7",
  "Color": "blue",
  "Size": 50,
  "Owner": "student",
  "Value": 300
}
```

That means:

- ✓ Asset exists
- ✓ Ledger update was successful
- ✓ Smart contract working correctly

A terminal window showing a series of chaincode commands and their outputs. The commands include 'peer chaincode invoke' to create an asset, 'peer chaincode query' to list all assets, and 'peer chaincode query' to read a specific asset. The outputs show the successful creation and retrieval of 'asset7' with a value of 300.

```
user@legion: /blockchain/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_CA --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_CA -c '{"function":"CreateAsset","Args":["asset7","blue",50,"student",300]}'
[chaincodeCmd] chaincodeInvokeQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset7","Color":"blue","Size":50,"Owner":"student","AppraisedValue":300}
user@legion: /blockchain/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset7","Owner":"student","Size":50}]
user@legion: /blockchain/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset7"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset7","Owner":"student","Size":50}
user@legion: /blockchain/fabric-samples/test-network$
```

What If We Get Error? If we see: Error: asset7 does not exist

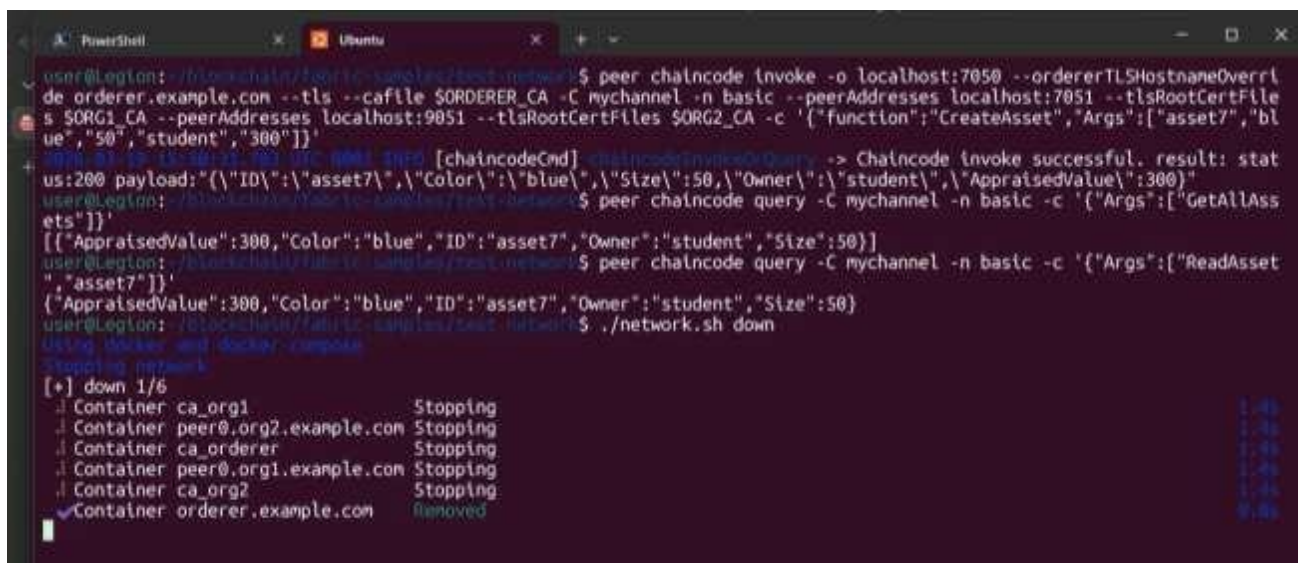
It means:

- Invoke failed earlier
- Transaction not committed
- We are using wrong channel or wrong organization context

STEP 10: Stop Network

./network.sh down

Stops all Docker containers.

A terminal window showing the execution of the 'down' script. It lists the containers to be stopped and shows their status as 'Stopping' or 'Removed'.

```
user@legion: /blockchain/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_CA --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_CA -c '{"function":"CreateAsset","Args":["asset7","blue",50,"student",300]}'
[chaincodeCmd] chaincodeInvokeQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset7","Color":"blue","Size":50,"Owner":"student","AppraisedValue":300}
user@legion: /blockchain/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset7","Owner":"student","Size":50}]
user@legion: /blockchain/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset7"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset7","Owner":"student","Size":50}
user@legion: /blockchain/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[*] down 1/6
  Container ca_org1 Stopping 1:40
  Container peer@org2.example.com Stopping 1:40
  Container ca_orderer Stopping 1:40
  Container peer@org1.example.com Stopping 1:40
  Container ca_org2 Stopping 1:40
  Container orderer.example.com Removed 0:00
```